Báo Cáo Bài Tập Lớn
Hệ Thống Điểm Danh Thông Minh

Nhóm: 9
24022454 Lê Việt Thành (Trưởng nhóm)
24022406 Nguyễn Nhật Minh
24022364 Tạ Duy Khánh
24022394 Nguyễn Đức Mạnh

Tháng 05/2026

Mục lục

Lời mở đầu **v**

1 Giới thiệu đề tài **vi**
1.1 Bối cảnh và động lực . . . vi
1.2 Mục tiêu . . . vi
1.3 Phạm vi hệ thống . . . vi
1.4 Đóng góp chính . . . vii

2 Phân tích yêu cầu **viii**
2.1 Tác nhân và bối cảnh sử dụng . . . viii
2.2 Yêu cầu chức năng . . . viii
2.3 Yêu cầu phi chức năng . . . viii
2.4 Sơ đồ use case . . . ix

3 Kiến trúc hệ thống **x**
3.1 Tổng quan kiến trúc . . . x
3.2 Sơ đồ component . . . x
3.3 Sơ đồ deployment . . . x
3.4 Mô hình lưu trữ dữ liệu . . . x

4 Thiết kế chi tiết **xii**
4.1 Mô hình dữ liệu . . . xii
4.1.1 Bảng users . . . xii
4.1.2 Bảng face_embeddings . . . xii
4.1.3 Bảng attendances . . . xii
4.2 Thiết kế API . . . xiii
4.3 Luồng đăng ký (Registration) . . . xiii
4.4 Luồng điểm danh (Attendance) . . . xiii
4.5 Sơ đồ sequence và activity . . . xiv
4.6 Công thức so sánh vector . . . xiv

5 Cài đặt hệ thống **xvii**
5.1 Backend (FastAPI) . . . xvii
5.2 Worker AI (Celery + DeepFace) . . . xvii
5.3 Redis và cache embedding . . . xvii
5.4 Qdrant . . . xviii
5.5 MinIO . . . xviii
5.6 Frontend . . . xviii
5.7 Nginx . . . xviii

6 Triển khai và vận hành **xix**
6.1 Docker Compose . . . xix
6.2 Quy trình chạy hệ thống . . . xix
6.3 Cấu hình biến môi trường . . . xix

7 Kiểm thử và đánh giá **xxi**
7.1 Kế hoạch kiểm thử . . . xxi
7.2 Bộ ca kiểm thử mẫu . . . xxi
7.3 Đánh giá hiệu năng . . . xxi

8 Hạn chế và hướng phát triển **xxii**
8.1 Hạn chế . . . xxii
8.2 Hướng phát triển . . . xxii

9 Kết luận **xxiii**

Danh sách hình vẽ

2.1 Sơ đồ use case của hệ thống ix

3.1 Sơ đồ thành phần (component diagram) xi
3.2 Sơ đồ triển khai (deployment diagram) xi

4.1 Sơ đồ sequence cho đăng ký và điểm danh xv
4.2 Sơ đồ activity cho luồng điểm danh xvi

Danh sách bảng

Lời mở đầu

Trong bối cảnh chuyển đổi số giáo dục và nhu cầu quản lý lớp học hiệu quả, điểm danh bằng khuôn mặt là một hướng đi khả thi giúp giảm thời gian thao tác thủ công, đồng thời cải thiện tính chính xác và minh bạch. Báo cáo này trình bày quá trình phân tích, thiết kế và triển khai hệ thống điểm danh thông minh dựa trên kiến trúc microservices và pipeline AI cho nhận dạng khuôn mặt.

Chương 1

Giới thiệu đề tài

1.1 Bối cảnh và động lực

Điểm danh thủ công bằng gọi tên hoặc quét thẻ có nhiều hạn chế như mất thời gian, dễ xảy ra nhầm lẫn, và khó kiểm soát trường hợp gian lận. Công nghệ thị giác máy tính và học sâu cho phép nhận diện khuôn mặt với độ chính xác cao, tạo nền tảng để tự động hóa quy trình điểm danh và tích hợp vào các hệ thống quản lý đào tạo.

1.2 Mục tiêu

Mục tiêu của dự án là xây dựng một hệ thống điểm danh thông minh có khả năng:

- Đăng ký khuôn mặt theo MSSV bằng ảnh hoặc video.
- Trích xuất và lưu trữ embedding khuôn mặt để đối sánh.
- Thực hiện điểm danh bằng ảnh chụp từ camera, trả kết quả nhanh.
- Lưu lịch sử điểm danh và hỗ trợ xuất báo cáo.
- Vận hành ổn định, dễ mở rộng và triển khai bằng Docker.

1.3 Phạm vi hệ thống

Hệ thống tập trung vào hai luồng chính: đăng ký và điểm danh. Ngoài ra, hệ thống hỗ trợ quản lý sinh viên, thống kê và xuất báo cáo. Các chức năng nâng cao như quản lý lớp học, lịch học theo ca, hoặc tích hợp SSO nằm ngoài phạm vi hiện tại.

1.4 Đóng góp chính

- Thiết kế kiến trúc microservices với backend, worker AI, và các dịch vụ dữ liệu độc lập.
- Xây dựng pipeline AI sử dụng DeepFace (ArcFace) và Qdrant để lưu trữ, truy vấn vector.
- Tích hợp MinIO để lưu ảnh, Redis cho cache và Celery làm hàng đợi tác vụ.
- Đóng gói triển khai bằng Docker Compose cho môi trường phát triển và triển khai.

Chương 2

Phân tích yêu cầu

2.1 Tác nhân và bối cảnh sử dụng

Hệ thống phục vụ hai nhóm người dùng chính:

- **Sinh viên**: đăng ký khuôn mặt, thực hiện điểm danh, xem lịch sử cá nhân.
- **Quản lý/Giảng viên**: quản lý sinh viên, xem báo cáo tổng hợp, xuất dữ liệu.

2.2 Yêu cầu chức năng

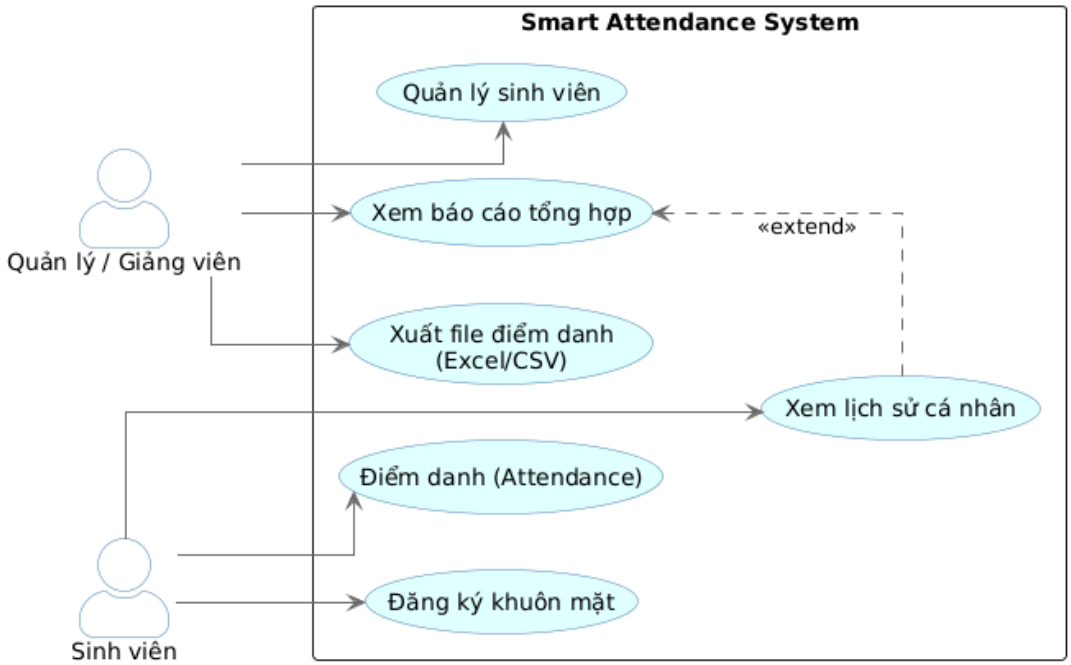
- Đăng ký khuôn mặt theo MSSV bằng ảnh (base64) hoặc video.
- Trích xuất embedding từ ảnh, tính master embedding và lưu vào Qdrant.
- Lưu thông tin sinh viên và log đăng ký vào MySQL.
- Lưu ảnh gốc vào MinIO để audit.
- Điểm danh bằng ảnh: nhận diện theo MSSV, trả kết quả matched/không matched.
- Cache embedding trên Redis để tăng tốc.
- Quản lý sinh viên: thêm/sửa/xóa/tra cứu.
- Xem lịch sử điểm danh và báo cáo tổng hợp.
- Xuất dữ liệu điểm danh ra CSV/Excel.

2.3 Yêu cầu phi chức năng

- Hiệu năng: phản hồi điểm danh trong vài giây.

- Độ tin cậy: retry task, xử lý lỗi worker rõ ràng.
- Bảo mật: hỗ trợ HTTPS, xác thực token, phân quyền theo vai trò.
- Tính sẵn sàng: service tự khởi động lại khi lỗi.
- Mở rộng: dễ scale backend/worker theo tải.
- Bảo trì: code tách lớp, có log và healthcheck.
- Tính tương thích: hỗ trợ trình duyệt hiện đại và thiết bị di động.
- Sao lưu: backup MySQL và MinIO định kỳ.

2.4 Sơ đồ use case



Hình 2.1: Sơ đồ use case của hệ thống

Chương 3

Kiến trúc hệ thống

3.1 Tổng quan kiến trúc

Hệ thống sử dụng kiến trúc microservices với các thành phần chính:

- **Nginx**: reverse proxy, phân luồng /api và phục vụ static UI.
- **Backend FastAPI**: cung cấp REST API, xử lý nghiệp vụ.
- **Worker Celery**: xử lý tác vụ AI nặng.
- **MySQL**: lưu trữ dữ liệu nghiệp vụ.
- **Redis**: cache embedding và làm message broker cho Celery.
- **Qdrant**: lưu trữ vector khuôn mặt và truy vấn tương đồng.
- **MinIO**: lưu trữ ảnh đăng ký và ảnh điểm danh.

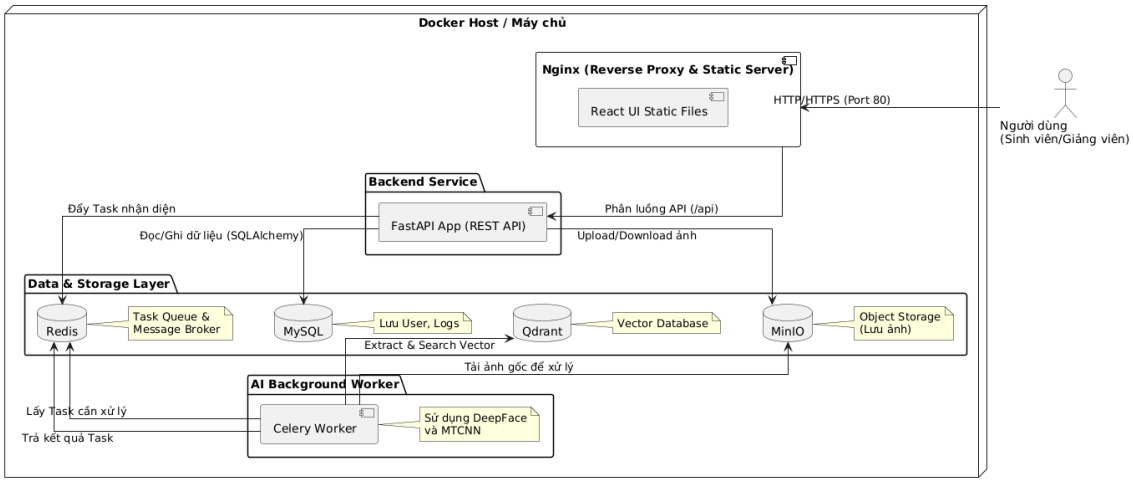
3.2 Sơ đồ component

3.3 Sơ đồ deployment

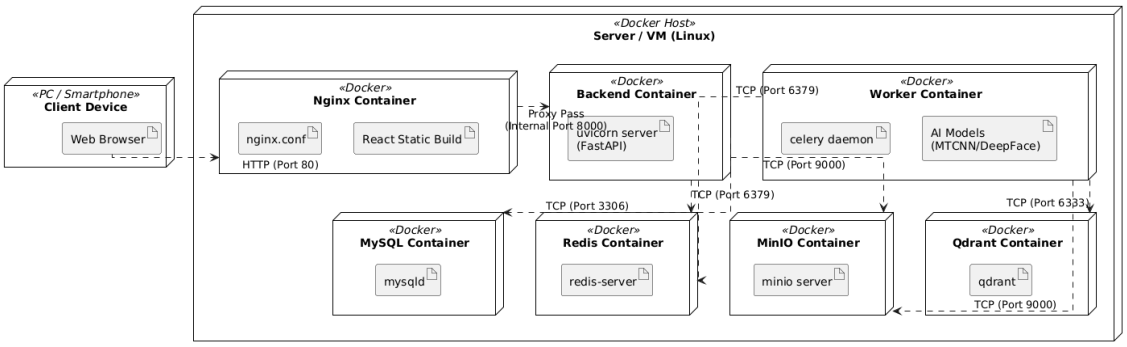
3.4 Mô hình lưu trữ dữ liệu

Hệ thống tách dữ liệu theo loại:

- **MySQL**: lưu user, embedding metadata, và lịch sử điểm danh.
- **Qdrant**: lưu vector embedding để truy vấn nhanh.
- **MinIO**: lưu ảnh gốc (đăng ký và điểm danh).
- **Redis**: cache embedding và làm broker cho Celery.



Hình 3.1: Sơ đồ thành phần (component diagram)



Hình 3.2: Sơ đồ triển khai (deployment diagram)

Chương 4

Thiết kế chi tiết

4.1 Mô hình dữ liệu

4.1.1 Bảng users

- `id`: khóa chính.
- `student_id`: mã sinh viên, duy nhất.
- `full_name`: họ tên sinh viên.
- `created_at`: thời điểm tạo.

4.1.2 Bảng face_embeddings

- `id`: khóa chính.
- `user_id`: khóa ngoại sang users.
- `qdrant_id`: ID vector trong Qdrant.
- `image_path`: đường dẫn ảnh trong MinIO.
- `created_at`: thời điểm tạo.

4.1.3 Bảng attendances

- `id`: khóa chính.
- `user_id`: khóa ngoại sang users.
- `check_in_time`: thời gian điểm danh.
- `confidence_score`: điểm tương đồng.

- `camera_id`: mã camera.
- `status`: trạng thái (success/failed/unknown).

4.2 Thiết kế API

Endpoint	Method	Mô tả
/health	GET	Kiểm tra trạng thái service.
/api/v1/members/register	POST	Đăng ký bằng danh sách ảnh base64.
/api/v1/members/register-video	POST	Đăng ký bằng video.
/api/v1/attendance/check-in	POST	Điểm danh bằng ảnh chụp.

4.3 Luồng đăng ký (Registration)

1. Client gửi danh sách ảnh hoặc video kèm MSSV và họ tên.
2. Backend tải ảnh lên MinIO và gọi Celery task trích xuất embedding.
3. Worker trích xuất vector qua DeepFace (ArcFace).
4. Backend tính master embedding bằng trung bình vector.
5. Lưu vector vào Qdrant, lưu metadata vào MySQL.
6. Cache embedding vào Redis.

4.4 Luồng điểm danh (Attendance)

1. Client gửi ảnh chụp từ camera kèm MSSV.
2. Backend kiểm tra MSSV và vector tồn tại.
3. Backend upload ảnh lên MinIO.
4. Backend gửi task nhận diện đến Worker (Celery).
5. Worker trích xuất vector từ ảnh và tìm kiếm Qdrant theo MSSV.
6. Backend nhận kết quả, lưu log điểm danh vào MySQL.

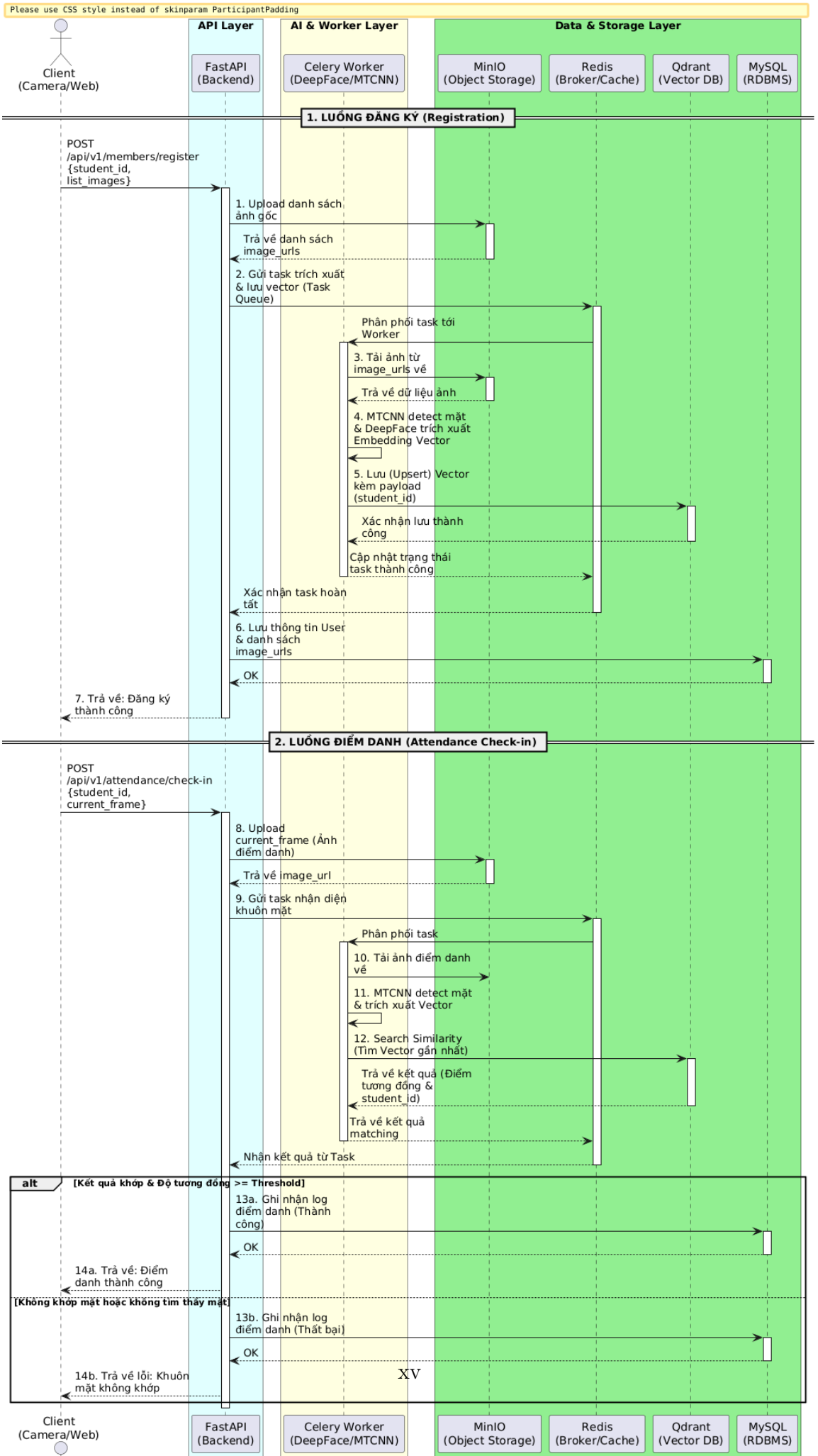
4.5 Sơ đồ sequence và activity

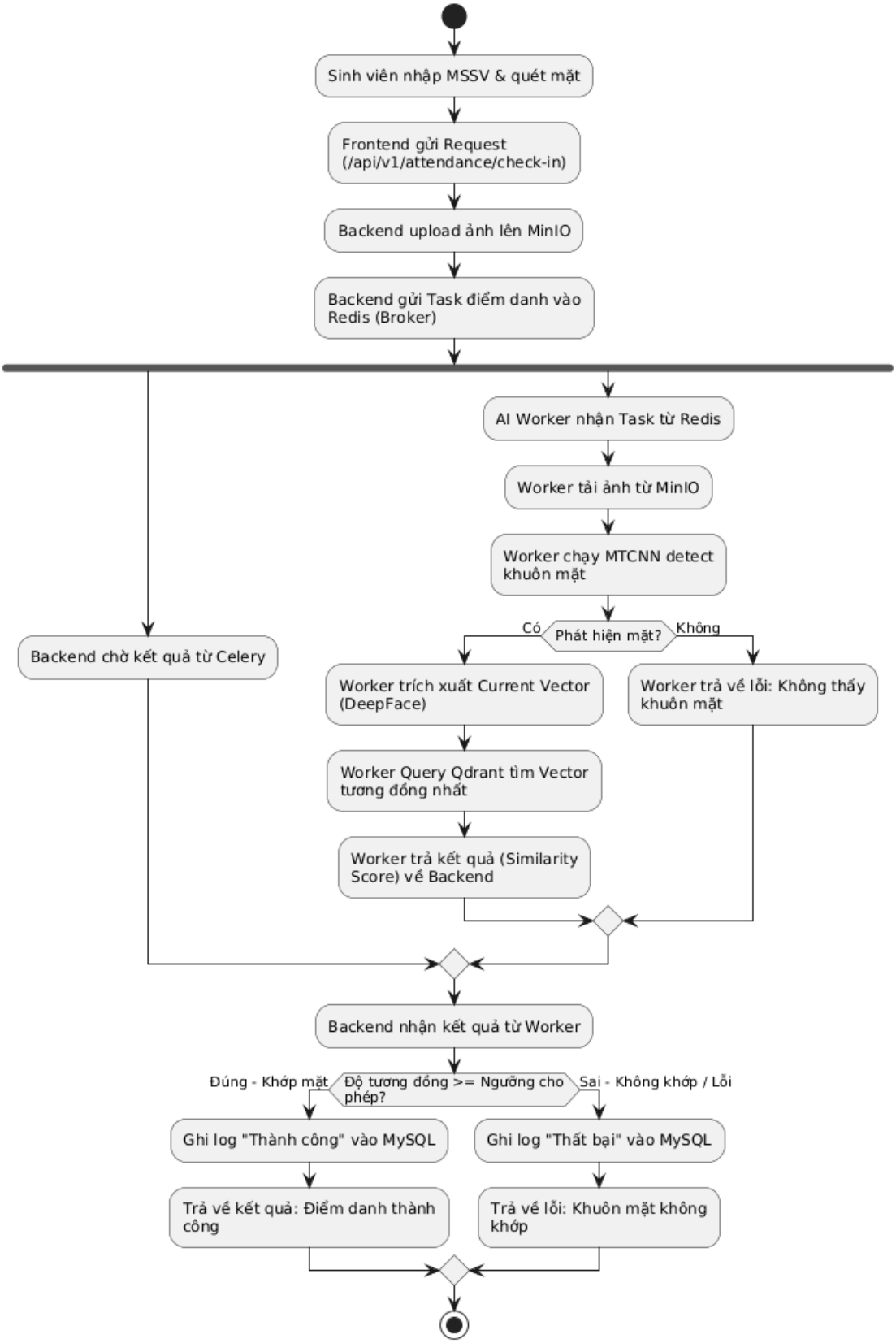
4.6 Công thức so sánh vector

Độ tương đồng cosine được tính như sau:

$$\text{cosine}(a, b) = \frac{a \cdot b}{\|a\|\|b\|}$$

Nếu điểm tương đồng lớn hơn ngưỡng `FACE_MATCH_THRESHOLD` thì kết quả được coi là khớp.


Please use CSS style instead of skinparam ParticipantPadding
Client (Camera/Web)
API Layer
FastAPI (Backend)
AI & Worker Layer
Celery Worker (DeepFace/MTCNN)
Data & Storage Layer
MinIO (Object Storage)
Redis (Broker/Cache)
Qdrant (Vector DB)
MySQL (RDBMS)
1. LUỒNG ĐĂNG KÝ (Registration)
POST /api/v1/members/register {student_id, list_images}
1. Upload danh sách ảnh gốc
Trả về danh sách image_urls
2. Gửi task trích xuất & lưu vector (Task Queue)
Phân phối task tới Worker
3. Tải ảnh từ image_urls về
Trả về dữ liệu ảnh
4. MTCNN detect mặt & DeepFace trích xuất Embedding Vector
5. Lưu (Upsert) Vector kèm payload (student_id)
Xác nhận lưu thành công
Cập nhật trạng thái task thành công
Xác nhận task hoàn tất
6. Lưu thông tin User & danh sách image_urls
OK
7. Trả về: Đăng ký thành công
2. LUỒNG ĐIỂM DANH (Attendance Check-in)
POST /api/v1/attendance/check-in {student_id, current_frame}
8. Upload current_frame (Ảnh điểm danh)
Trả về image_url
9. Gửi task nhận diện khuôn mặt
Phân phối task
10. Tải ảnh điểm danh về
11. MTCNN detect mặt & trích xuất Vector
12. Search Similarity (Tìm Vector gần nhất)
Trả về kết quả (Điểm tương đồng & student_id)
Trả về kết quả matching
Nhận kết quả từ Task
alt
[Kết quả khớp & Độ tương đồng >= Threshold]
13a. Ghi nhận log điểm danh (Thành công)
OK
14a. Trả về: Điểm danh thành công
[Không khớp mặt hoặc không tìm thấy mặt]
13b. Ghi nhận log điểm danh (Thất bại)
OK
14b. Trả về lỗi: Khuôn mặt không khớp
Client (Camera/Web)
FastAPI (Backend)
Celery Worker (DeepFace/MTCNN)
MinIO (Object Storage)
Redis (Broker/Cache)
Qdrant (Vector DB)
MySQL (RDBMS)



Hình 4.2: Sơ đồ activity cho luồng điểm danh

Chương 5

Cài đặt hệ thống

5.1 Backend (FastAPI)

Backend triển khai bằng FastAPI, gồm các tầng:

- **API Router**: định nghĩa endpoint đăng ký và điểm danh.
- **Service Layer**: xử lý nghiệp vụ (MemberService).
- **Database Layer**: SQLAlchemy ORM.

CORS được mở cho mọi nguồn để tiện tích hợp frontend trong giai đoạn phát triển.

5.2 Worker AI (Celery + DeepFace)

Worker xử lý hai tác vụ chính:

- `extract_face_embedding_task`: trích xuất embedding khi đăng ký.
- `recognize_face_task`: nhận diện khi điểm danh.

Worker sử dụng DeepFace với model ArcFace và detector MTCNN. Qdrant được dùng để lưu và truy vấn vector.

5.3 Redis và cache embedding

Redis được dùng làm broker cho Celery và lưu cache embedding. Mỗi embedding được lưu với key dạng `user:{student_id}:embedding` và có TTL cấu hình.

5.4 Qdrant

Qdrant lưu vector embedding theo cấu trúc point có payload `student_id` và `user_id`. Khi nhận diện, worker truy vấn Qdrant với filter theo `student_id` và lấy kết quả gần nhất.

5.5 MinIO

MinIO lưu ảnh gốc để audit và phục vụ truy xuất khi cần. Backend upload ảnh và nhận URL để truyền cho worker xử lý.

5.6 Frontend

Frontend xây dựng bằng Vue, cung cấp các màn hình chính: đăng ký, điểm danh, dashboard, thống kê và quản trị. Ứng dụng được build thành static files và phục vụ bởi Nginx.

5.7 Nginx

Nginx thực hiện reverse proxy cho API (đường dẫn /api) và phục vụ static UI tại /.

Chương 6

Triển khai và vận hành

6.1 Docker Compose

Hệ thống được đóng gói bằng Docker Compose với các service:

- mysql, redis, qdrant, minio
- backend (FastAPI)
- worker (Celery)
- nginx (reverse proxy + static UI)

6.2 Quy trình chạy hệ thống

```
docker compose build
docker compose up -d
docker compose logs -f
```

6.3 Cấu hình biến môi trường

Các biến môi trường chính:

Biến	**Mô tả**
DATABASE_URL	Chuỗi kết nối MySQL.
REDIS_URL	Chuỗi kết nối Redis.
CELERY_BROKER_URL	Broker cho Celery.
CELERY_RESULT_BACKEND	Backend kết quả Celery.
QDRANT_URL	Địa chỉ Qdrant.

MINIO_ENDPOINT	Địa chỉ MinIO.
MINIO_ACCESS_KEY	Access key MinIO.
MINIO_SECRET_KEY	Secret key MinIO.
FACE_MATCH_THRESHOLD	Ngưỡng nhận diện khuôn mặt.
DEEPFACE_MODEL_NAME	Model nhận diện (mặc định ArcFace).
FACE_DETECTOR_BACKEND	Backend detector (mặc định mtcnn).

Chương 7

Kiểm thử và đánh giá

7.1 Kế hoạch kiểm thử

Hệ thống được kiểm thử theo ba nhóm chính:

- **Kiểm thử chức năng**: đăng ký, điểm danh, báo cáo.
- **Kiểm thử hiệu năng**: đo thời gian phản hồi đăng ký và điểm danh.
- **Kiểm thử độ ổn định**: kiểm tra retry khi worker lỗi hoặc timeout.

7.2 Bộ ca kiểm thử mẫu

Ca kiểm thử	Đầu vào	Kết quả mong đợi
Đăng ký hợp lệ	3 ảnh rõ mặt	Tạo embedding và trả thành công.
Đăng ký thiếu ảnh	1 ảnh	Báo lỗi thiếu ảnh.
Điểm danh hợp lệ	1 ảnh rõ mặt, MSSV tồn tại	Trả matched = true.
Điểm danh sai người	Ảnh người khác	Trả matched = false.
Không phát hiện mặt	Ảnh không có mặt	Báo lỗi chất lượng.

7.3 Đánh giá hiệu năng

Mục tiêu hiệu năng đặt ra là thời gian phản hồi cho điểm danh trong vài giây. Thời gian xử lý phụ thuộc vào tốc độ tải ảnh, hiệu năng của worker AI, và thời gian truy vấn Qdrant. Với kiến trúc hiện tại, hệ thống có thể mở rộng bằng cách tăng số lượng worker và tối ưu mô hình AI.

Chương 8

Hạn chế và hướng phát triển

8.1 Hạn chế

- Phụ thuộc vào chất lượng ảnh đầu vào.
- Thời gian đăng ký có thể lâu nếu nhiều ảnh/video.
- Chưa có cơ chế phân quyền chi tiết cho nhiều vai trò.

8.2 Hướng phát triển

- Bổ sung cơ chế nhận diện đa khuôn mặt trong một khung hình.
- Tích hợp kiểm tra chống giả mạo (liveness detection).
- Hỗ trợ phân quyền chi tiết theo lớp học hoặc môn học.
- Tích hợp dashboard phân tích dữ liệu theo thời gian thực.

Chương 9

Kết luận

Hệ thống điểm danh thông minh đã được thiết kế và triển khai theo kiến trúc microservices, tận dụng pipeline AI để nhận diện khuôn mặt. Kết quả cho thấy hệ thống đáp ứng đầy đủ yêu cầu chức năng, có khả năng triển khai và mở rộng. Trong tương lai, hệ thống có thể được cải tiến thêm về bảo mật, hiệu năng và trải nghiệm người dùng.

Tài liệu tham khảo

[1] DeepFace: https://github.com/serengil/deepface

[2] Qdrant Vector Database: https://qdrant.tech/

[3] FastAPI: https://fastapi.tiangolo.com/

[4] Celery: https://docs.celeryq.dev/